Integration Strategy Analysis: Reconciling the netcfg Compiler with the Plan Topology Model

A Comparative Assessment of Backport, Rewrite, and Hybrid Approaches

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026

Abstract. Two bodies of work address the same vision of declarative, vendor-neutral network configuration: (i) **netcfg**, a production Rust compiler with 16 CLI commands and 23 primitives; and (ii) a formal **Narrow-Waist Architecture** establishing Plan Topologies (G_{plan}) and Functorial Overlay Mappings. This report analyses three strategies for reconciling the two: incremental backport (Option A), clean-slate reimplementation (Option B), and a two-layer hybrid (Option C) that establishes a formal **Structural Composition** pipeline while retaining netcfg as the realisation engine. Each option is evaluated against implementation effort, theoretical fidelity, and thesis alignment using a weighted decision matrix. The analysis recommends a phased migration to the two-layer architecture, enabling absolute **Vendor Swap** capabilities and mathematically verified **Intent Drift** detection.

Contents

Contents

1 Introduction

1.1 Problem Statement

Two implementations of the same research vision now exist at markedly different maturity levels. The **netcfg** compiler is a production-grade Rust codebase that compiles declarative blueprints into vendor-specific configurations via a four-stage pipeline (parse → execute → diff → render). It supports 23 primitives across five tiers (topology, allocation, policy, overlay, meta), 16 CLI commands, and has been validated against multi-site case studies.

In parallel, the formal model described in ANK-TR-2026-03 and ANK-TR-2026-04 establishes a mathematically rigorous foundation: plan topologies $G_{\text{plan}} = (V, E, E_{\text{conn}}, E_{\text{own}}, \pi_V, \pi_E)$, functorial overlay mappings $\mathcal{F}(G_{\text{plan}}) = \bigoplus_{\ell \in L} \mathcal{F}_\ell(G_{\text{plan}})$, a supra-adjacency tensor for multilayer coupling, DPO graph rewriting for topology mutations, and an exhaustive protocol catalogue with 58 attribute prefixes.

The question this report addresses: *how should these two artefacts be reconciled?* The answer has implications for the thesis contribution, for ongoing development velocity, and for the practical utility of the toolchain.

1.2 The What and the Why: Mathematical Elegance vs Operational Reality

To understand the core conflict, one must contrast the foundational mathematics (The What) with the operational imperative (The Why). The formal model is a pristine algebraic environment: networks are immutable multilayer graphs, state is purely flat, and configuration overlays are deterministically extracted via Functors. This model guarantees *correctness by construction* and serves as the definitive **Narrow-Waist** for network state. It acts as an **Anchor of Reality** that allows us to mathematically calculate **Intent Drift**.

However, the reality of network engineering is inherently stateful, messy, and bound to vendor-specific text templates that are in constant flux. The **netcfg** compiler has succeeded in the real world precisely because it embraces this messiness—providing imperative orchestration, translating visual **Structural Composition** intent into concrete steps, and robustly rendering vendor syntax. Yet, lacking the strict mathematical bounds of the formal model, its codebase risks becoming brittle under the weight of accumulating network features. Reconciling the two is not merely an engineering task; it is the fundamental research challenge of bridging a formal tensor algebra to an imperative execution runtime, ultimately protecting the operator's intellectual property via seamless **Vendor Swap** capabilities.

1.3 Scope

This report provides:

1. A side-by-side architectural comparison of netcfg and the formal model.
2. A conceptual alignment analysis identifying compatible, divergent, and missing concepts.
3. Three candidate integration strategies, each with detailed pros, cons, effort estimates, and risk assessments.
4. A multi-criteria decision matrix producing a ranked recommendation.
5. Concrete migration pathways with decision gates.

1.4 Structure

Section 2 summarises each system. Section 3 contextualises the architecture against related work. Section 4 analyses conceptual alignment and gaps. Sections 5–7 detail the three options. Section 8 presents the comparative evaluation. Section 9 describes migration pathways. Section 10 concludes. Appendix A provides a formal concept mapping; Appendix B catalogues protocol coverage gaps.

2 System Summaries

2.1 The netcfg Compiler

Language and ecosystem. netcfg is implemented in Rust, organised as a Cargo workspace with three crates: `netcfg` (CLI binary), `netcfg-core` (library), and `deviceir-contract` (serialisation). The graph backend is provided by the sibling `ank_nte` crate.

Pipeline. Compilation proceeds in four stages:

1. **Parse** — The structural specification aggregated via the Structural Composition strategy is codified into a YAML blueprint, which is parsed into a `DesignPlan` AST via `dsl/parser.rs` and `dsl/schema.rs`.
2. **Execute** — Layer-by-layer primitive execution via `executor/runner.rs`; each primitive mutates the NTE topology and accumulates per-device state in `DeviceContext` structs.
3. **Diff** — `DiffEngine` computes a `MutationPlan` by comparing pre- and post-execution topologies, keyed by semantic node and edge identifiers.
4. **Render** — MiniJinja templates in `render/engine.rs` produce vendor-specific configuration text from `DeviceIR` (the frozen, immutable form of `DeviceContext`).

Primitives. 23 primitives are organised into five tiers:

Tier	Count	Examples
Topology	2	`mesh_nodes`, `build_protocol_layer`
Allocation	4	`provision_ips`, `define_ip_pool`
Policy	8	`policy_routing`, `build_zone_policy`, `build_nat_policy`, `build_qos_policy`
Overlay	2	`overlay_vxlan`, `overlay_evpn`
Meta	7	`conditional`, `assert_state`, `tag_nodes`, `wasm_plugin`

Data model. `DeviceContext` is a per-device mutable struct with 25+ fields spanning IPAM assignments (`src_ip`, `dst_ip`, `subnet`, `loopback0`), interface state (`Vec<InterfaceEndpoint>`), overlay state (`vtep_ip`, `bgp_as`), untyped policy stanzas (`Vec<Stanza>`), typed security and QoS models, and a metadata map. After all primitives execute, `freeze()` produces an immutable `DeviceIR` with typed `StandardStanza` variants.

CLI. 16 commands: `generate`, `validate`, `plan`, `inspect`, `lint`, `describe`, `fmt`, `impact`, `ci_run`, `drift`, `explain`, `import`, `report`, `schema`, `sync`, `export_clab`.

Known gaps. The architecture corrections analysis (v7.5) identified five structural deviations from the thesis model: absence of an edge-endpoint model (Phase 185a, now addressed), missing edge properties, manual primitive ordering, absent protocol stanza derivation, and broken NQL group expansion.

2.2 The Plan Topology Model (ANK-TR-2026-03/04)

Plan topology. The source of truth is a single graph $G_{\text{plan}} = (V, E, E_{\text{conn}}, E_{\text{own}}, \pi_V, \pi_E)$ where V contains devices, E contains endpoints (interfaces), $E_{\text{own}} \subseteq V \times E$ expresses device ownership, and $E_{\text{conn}} \subseteq E \times E$ expresses physical connectivity. All protocol state is stored as flat key–value attributes on nodes (π_V) or endpoints (π_E), never on edges.

Attribute grammar. A formal EBNF grammar governs attribute naming: `protocol_prefix "_" field_name`, with 58 registered protocol prefixes spanning L1–L7. Ten primitive types (bool, u32, u64, f64, string, ipv4_addr, ipv6_addr, ip_prefix, mac_addr, enum) and three collection flattening rules (ordered lists → indexed keys, sets → CSV strings, maps → embedded neighbour keys).

Functorial overlay mapping. Each protocol layer ℓ defines a functor $\mathcal{F}_\ell$ that extracts a subgraph from G_{plan} by filtering on attribute prefix. The full overlay is the multilayer composition $\mathcal{F}(G_{\text{plan}}) = \bigoplus_\ell \mathcal{F}_\ell(G_{\text{plan}})$ with identity couplings linking the same physical device across layers. A dependency DAG orders functor evaluation: $\mathcal{F}_{\text{PHY}} \to \mathcal{F}_{\text{L2}} \to \mathcal{F}_{\text{STP}} \to \mathcal{F}_{\text{MPLS}} \to \{\mathcal{F}_{\text{OSPF}} \mid \mathcal{F}_{\text{IS-IS}}\} \to \mathcal{F}_{\text{BGP}} \to \mathcal{F}_{\text{EVPN}}$.

Supra-adjacency tensor. The multilayer structure is represented as a rank-4 tensor $M^{i\alpha}_{j\beta}$ (or equivalently an $NL \times NL$ block matrix) encoding both intra-layer adjacencies ($\alpha = \beta$) and inter-layer couplings ($\alpha \neq \beta$).

DPO graph rewriting. Topology mutations are expressed as double-pushout rules $p : L \xleftarrow{l} K \xrightarrow{r} R$, where L is the pattern, K the glue graph, and R the replacement. This guarantees well-defined rewrites with no dangling edges.

Three-phase validation.

1. **Structural constraints** — unique ownership, no self-loops, bidirectional connectivity, no orphan nodes. $O(|V| + |E|)$.
2. **Protocol consistency** — timer agreement, peer reciprocity, VLAN overlap, IP uniqueness, label range separation.
3. **Completeness** — every active protocol layer has all required attributes populated. $O(|V| \cdot |A|)$.

Protocol catalogue. ANK-TR-2026-04 catalogues 58 protocol attribute prefixes across three tiers and seven OSI layers, with per-attribute type annotations, validation rules, and design patterns for common topologies (full mesh, route reflector, EVPN fabric variants).

2.3 Side-by-Side Feature Matrix

Capability	netcfg	Formal Model
Source of truth	NTE graph + DeviceContext structs	Plan topology G_{plan}
Attribute schema	Ad-hoc Rust fields + metadata map	Formal EBNF grammar, 58 prefixes
Per-interface state	`InterfaceEndpoint` (Phase 185a)	First-class endpoints in E
Multilayer model	DAG of overlay graphs (NTE layers)	Supra-adjacency tensor $M_{j\beta}^{i\alpha}$
Overlay extraction	`build_protocol_layer` primitive	Functorial extraction $\mathcal{F}_\ell$
Topology mutation	Imperative primitive execution	DPO graph rewriting rules
Dependency ordering	Manual layer ordering in YAML	Formal DAG with functor laws
Validation	Assertions + lint + shadow rules	Three-phase (structural, protocol, completeness)
Protocol coverage	~12 protocols (routing, overlay, security)	58 protocol prefixes, three tiers
Configuration render	MiniJinja templates per vendor OS	Out of scope (compiler concern)
CLI tooling	16 commands	N/A (theoretical model)
MVCC transactions	No versioning; single-write	Snapshot isolation, atomic commits
Implementation	~20k lines Rust	Specification only (Python/Polars prototype)

3 Related Work and Architectural Context

The architectural challenge of reconciling an orchestration engine with a formal graph model intersects several distinct areas of networking research and software engineering.

3.1 Intent-Based Networking and Orchestration

Early Intent-Based Networking (IBN) architectures, such as Lumi [0], proposed tiered translations from high-level intent to device configurations. Commercial IBN controllers (e.g., Juniper Apstra, Cisco DNA Center) implement proprietary graph-like data stores to maintain intended versus operational state. However, these systems inextricably couple the orchestration logic, the data model, and the device templating. The two-layer architecture proposed in this report seeks to explicitly decouple the formal mathematical data model (G_{plan}) from the volatile orchestration and templating logic (`netcfg`). This advances the IBN paradigm by ensuring the source of truth is a mathematically verifiable artefact rather than an opaque operational database.

3.2 Formal Network Models and Verification

Formal network modelling frequently relies on post-hoc verification, where tools like Batfish [0] or Minesweeper [0] parse existing device configurations into a vendor-neutral intermediate representation for analysis. NetKAT [0] provides a rigorous algebra for forwarding-plane policies. Unlike these post-hoc analysers, the integration strategy presented here pursues a *constructive* approach: the formal model (equipped with DPO graph rewriting and functorial extraction) acts as the engine generating the configuration. By enforcing invariants at the time of construction, the architecture prevents structural errors before vendor templates are ever rendered.

3.3 Relational and Graph-Native Network Schemas

Historically, network configuration systems have treated device configurations as document trees (e.g., YANG/OpenConfig models [0], JSON/YAML dictionaries). While RFC 8345 [0] defines a standard for network topologies, it remains largely structural rather than algebraic. Moving the source of truth to a dynamic, Graph-Native Columnar Attribute Store represents a significant departure from Array-of-Structs (AoS) document models. This relates closely to advancements in Data-Oriented Design (DOD) and columnar databases, translating techniques originally developed for data science into a formal repository for network state. This shift addresses the performance and rigidity issues common in traditional document-based configuration pipelines.

4 Conceptual Alignment Analysis

4.1 Compatible Concepts

Several formal concepts map cleanly onto existing netcfg structures, requiring adaptation rather than invention.

Attribute naming grammar ↔ DeviceContext metadata. The formal EBNF grammar (`protocol_prefix "_" field_name`) can be adopted as a naming convention for the existing `metadata: HashMap<String, JsonValue>` on `DeviceContext`. No structural change is needed; the grammar becomes a validation rule enforced at the DSL layer.

Endpoint model ↔ InterfaceEndpoint. Phase 185a introduced `InterfaceEndpoint` with per-edge state (address, subnet, VRF, peer, cost, bandwidth, media type). This is structurally equivalent to the formal model's endpoint set $E(v)$ with ownership edges E_{own}. The gap is that `InterfaceEndpoint` lives inside `DeviceContext` rather than as first-class graph entities; closing this gap requires promoting endpoints to NTE nodes.

Three-phase validation ↔ assertion system. netcfg's `assert_state` primitive, `lint` command, and shadow topology rules collectively cover much of the formal validation model. Structural constraints (Phase 1) map to shadow rules; protocol consistency (Phase 2) maps to assertions with CEL predicates; completeness (Phase 3) maps to template preflight checks. The gap is systematisation: the formal model defines validation as an exhaustive, ordered pipeline rather than ad-hoc user-authored rules. To bridge this, Option C implements formal **Assertions** as native Rust traits evaluating against the columnar store. For example, verifying that MACsec keys match across a link ($e \in E_{\text{conn}}$) translates from theoretical math into an executable CEL-backed graph query:

```rust
// Phase 2 Protocol Consistency: MACsec symmetric key validation
fn evaluate_assertion(graph: &PlanTopology, edge: &Edge) -> Result<(), Error> {
    let src_key = graph.get_attribute(edge.src, "macsec_ckn")?;
    let dst_key = graph.get_attribute(edge.dst, "macsec_ckn")?;
    if src_key != dst_key {
        return Err(Error::IntentDrift(
            format!("Assertion Failed: MACsec mismatch on edge {:?}", edge.id)
        ));
    }
    Ok(())
}
```

Protocol catalogue ↔ attribute schema. The 58-prefix catalogue can serve as the schema for a metadata registry (Phase 301 in the roadmap), validating that attribute keys conform to the grammar and carry correct types.

Functor dependency DAG ↔ primitive ordering. The formal DAG ($\mathcal{F}_{\text{PHY}} \to \mathcal{F}_{\text{L2}} \to \cdots \to \mathcal{F}_{\text{EVPN}}$) directly addresses Phase 194 (primitive ordering validation). Adopting the DAG as a compile-time check on layer ordering in the YAML blueprint is straightforward.

4.2 Divergent Concepts

Four areas exhibit fundamental architectural differences that cannot be bridged by incremental adaptation alone.

Polars DataFrames vs Rust structs. The formal model's NTE backend stores all attributes in columnar Polars DataFrames, enabling bulk vectorised operations, lazy evaluation, and schema-on-read flexibility. netcfg uses hand-written Rust structs (`DeviceContext`, `InterfaceEndpoint`) with fixed fields, providing compile-time type safety but precluding dynamic schema evolution. Bridging this requires either (a) moving netcfg to a DataFrame-backed store, losing compile-time guarantees, or (b) generating Rust types from the attribute catalogue, adding build complexity.

DPO rewriting vs imperative mutation. netcfg primitives are imperative functions that mutate the NTE topology and `DeviceContext` in-place. The formal model specifies DPO rules that guarantee well-defined rewrites via pushout diagrams. Retrofitting DPO semantics onto imperative primitives would require wrapping every mutation in a match–glue–replace transaction, fundamentally changing the execution model.

Supra-adjacency tensor vs DAG-of-graphs. The formal model's rank-4 tensor provides a unified algebraic representation enabling spectral analysis, path queries across layers, and formal coupling verification. netcfg maintains layers as separate NTE graphs with implicit coupling through shared node hostnames. The tensor formalism is analytically powerful but computationally expensive for the configuration generation use case; it is more relevant for verification and reachability analysis.

Functorial composition vs linear pipeline. In the formal model, functors compose categorically: $\mathcal{F}_\ell \circ \mathcal{F}_k$ preserves identity and associativity laws. netcfg's pipeline is a linear sequence of primitive invocations where ordering is specified by the user in YAML. The formal approach enables algebraic reasoning about composition (e.g., proving that reordering commutative functors yields identical output); the imperative approach offers pragmatic flexibility but no compositional guarantees.

4.3 Gap Analysis

Concept	netcfg	Model	Effort to Bridge
Plan topology G_{plan}	Partial	Full	Medium — promote endpoints to NTE nodes
Endpoint ownership E_{own}	Implicit	Full	Low — already in InterfaceEndpoint
Connectivity edges E_{conn}	Partial	Full	Medium — endpoint-to-endpoint edges
Attribute grammar	None	Full	Low — validation rule on metadata
Attribute type system	Ad-hoc	Full	Medium — schema registry
Collection flattening	None	Full	Low — serialisation convention
Functor extraction $\mathcal{F}_\ell$	Partial	Full	Medium — refactor `build_protocol_layer`
Functor composition laws	None	Full	High — requires algebraic framework
Dependency DAG	None	Full	Low — static analysis of layer specs
DPO rewriting	None	Full	High — new execution model
Supra-adjacency tensor	None	Full	High — new data structure
Three-phase validation	Partial	Full	Medium — systematise existing checks
MVCC transactions	None	Full	High — snapshot isolation in NTE
Protocol catalogue (58 prefixes)	Partial	Full	Medium — schema + attribute registry
Design patterns (RR, EVPN, etc.)	None	Full	Medium — template library

5 Option A: Incremental Backport

5.1 Description and Scope

Option A retains the existing netcfg codebase and incrementally adopts formal concepts where they add value, without restructuring the execution model. The formal model serves as a *design guide* rather than an implementation target: concepts like the attribute grammar, dependency DAG, and three-phase validation are backported as features; concepts like DPO rewriting and the supra-adjacency tensor are documented as future work.

5.2 Detailed Pros

1. **Preserves 18 months of engineering.** All 23 primitives, 16 CLI commands, case study validations, and integration tests remain intact.
2. **Incremental delivery.** Each backported concept can be shipped independently (attribute grammar in one release, dependency DAG in the next), maintaining continuous usability.
3. **Low risk to existing users.** The Python frontend (`ank_pydantic`) and existing blueprints continue to work throughout the migration.
4. **Proven architecture.** The four-stage pipeline has been validated against real topologies; backporting concepts onto a working system avoids second-system effect.
5. **Thesis-compatible.** The architecture corrections (v7.5) already demonstrate convergence toward the formal model; continuing this trajectory is a natural thesis narrative.

5.3 Detailed Cons

1. **Structural ceiling.** Some formal concepts (DPO rewriting, functorial composition laws) cannot be meaningfully backported without rewriting the execution model.
2. **Attribute sprawl.** Without a columnar store, supporting 58 protocol prefixes means either (a) adding 200+ fields to `DeviceContext` or (b) relying on the untyped metadata map, losing compile-time safety.
3. **Multilayer coupling remains implicit.** Layers are separate NTE graphs with hostname-based coupling; formal inter-layer analysis (spectral properties, cross-layer path queries) is not achievable.
4. **Theoretical fidelity limited.** The thesis cannot claim full implementation of the formal model; it must carefully delineate which concepts are implemented and which are specified only.
5. **Technical debt accumulation.** Each backported concept must integrate with existing code paths, increasing coupling and making future refactoring harder.

5.4 Effort Estimate and Phasing

Phase	Scope	Estimate
B1	Attribute grammar validation (schema registry)	1–2 weeks
B2	Promote endpoints to NTE nodes	2–3 weeks
B3	Dependency DAG + ordering validation	1 week
B4	Systematise three-phase validation	2–3 weeks
B5	Protocol catalogue integration	2–3 weeks
B6	Design pattern library	2–4 weeks
	Total	**10–16 weeks**

5.5 Risk Assessment

Integration risk (Medium): Each backported concept touches core data structures; regressions in existing primitives are possible.

Scope creep (Medium): Once formal concepts are partially adopted, the temptation to "complete" the adoption may expand scope unboundedly.

Thesis coherence (Low): The incremental narrative is defensible provided the report clearly delineates implemented vs specified concepts.

5.6 What Gets Lost

Concepts that cannot be meaningfully backported:

- DPO rewriting guarantees (no-dangling-edge property).
- Functorial composition laws (identity, associativity proofs).
- Supra-adjacency tensor and spectral analysis.
- MVCC snapshot isolation.
- Algebraic reasoning about pipeline commutativity.

6 Option B: Clean-Slate Reimplementation

6.1 Description and Scope

Option B implements the formal model from scratch, using ANK-TR-2026-03/04 as the specification. The new system would use Polars DataFrames for attribute storage, implement DPO rewriting for topology mutations, represent the multilayer structure as a supra-adjacency tensor, and enforce functorial composition laws. The existing netcfg CLI and template rendering would be reimplemented as a thin layer atop the new engine.

6.2 Detailed Pros

1. **Full theoretical fidelity.** Every formal concept is implemented as specified; the thesis can claim a complete reference implementation.
2. **Clean architecture.** No legacy constraints; the codebase can be structured to mirror the formal model's decomposition exactly.
3. **Extensibility.** New protocols are added by registering attribute schemas and functor rules, not by writing Rust primitives.
4. **Verification-ready.** The supra-adjacency tensor enables cross-layer reachability analysis and spectral diagnostics from day one.
5. **MVCC semantics.** Proper snapshot isolation enables safe concurrent evaluation and rollback.

6.3 Detailed Cons

1. **18 months of work discarded.** All primitives, CLI commands, tests, and case study validations must be reimplemented.
2. **Second-system effect.** The temptation to over-engineer the new system based on theoretical elegance rather than practical need is substantial.
3. **Time-to-value.** A clean-slate rewrite must reach feature parity with netcfg before it becomes useful; this is a long runway with no intermediate deliverables.
4. **Language decision.** The formal model prototypes use Python/Polars; reimplementing in Rust requires porting the DataFrame backend or introducing a Python dependency, both of which add complexity.
5. **Thesis timeline risk.** A rewrite of this scope likely exceeds the available thesis timeline, resulting in an incomplete implementation that is worse than either the formal model or netcfg alone.

6.4 Effort Estimate and Phasing

Phase	Scope	Estimate
R1	Plan topology engine (graph + attributes + validation)	6–8 weeks
R2	Functor framework + dependency DAG	4–6 weeks
R3	DPO rewriting engine	4–6 weeks
R4	Protocol catalogue + schema registry	3–4 weeks
R5	CLI reimplementation (16 commands)	4–6 weeks
R6	Template rendering + vendor transforms	2–3 weeks
R7	Case study validation + regression tests	3–4 weeks
	Total	**26–37 weeks**

6.5 Risk Assessment

Timeline risk (High): 6–9 months is likely to exceed the thesis submission window.

Feature parity risk (High): Many netcfg features (WASM plugins, conditional execution, CI integration) are difficult to replicate.

Integration risk (Medium): The Python frontend must be ported to a new API surface.

Motivation risk (Medium): Reimplementing solved problems is demoralising and diverts energy from novel contributions.

6.6 What Gets Preserved from netcfg

Only high-level design knowledge transfers:

- Blueprint DSL syntax (can be reused with a new parser backend).
- MiniJinja template library (vendor-specific templates are reusable).
- Case study YAML files (test inputs remain valid if DSL is preserved).
- Architectural lessons (e.g., the need for edge-endpoint models, the importance of primitive ordering).

7 Option C: Two-Layer Architecture

7.1 Description

Option C restructures the system into two layers, as illustrated in Figure 1:

1. **NTE plan topology engine** — implements the formal model's plan topology, attribute schema, functorial extraction, and validation pipeline as a library (in `ank_nte` or a new crate). To bridge the gap between the formal model's Polars DataFrames and Rust's static typing, this engine will implement a lightweight, dynamically-typed Columnar Attribute Store (employing a Struct of Arrays memory layout). This provides schema-on-read flexibility without the overhead and compile-time penalty of a full data-science dependency like `polars-rs`.
2. **netcfg orchestration layer** — retains the existing CLI, primitives, and template rendering, but delegates graph operations, attribute storage, and validation to the plan topology engine.

The key insight is that netcfg's value lies in its *orchestration* (primitives, pipeline sequencing, CLI UX, vendor transforms) while the formal model's value lies in its *data model and algebra* (plan topology, functors, validation). Option C assigns each to the layer where it belongs.

7.2 Architectural Deep-Dive: Columnar Attributes and Graph Properties in `ank_nte`

The core technical shift in Option C is upgrading the `ank_nte` graph engine to use a Columnar Attribute Store natively linked to its graph edges. This resolves the divergence between the formal model's mathematical elegance and Rust's static typing constraints.

As illustrated in Figure 3, this approach fundamentally changes how network state is stored and queried:

1. **From Structs to Attribute Vectors:** Currently, `netcfg` allocates a monolithic `DeviceContext` struct for every node, leading to sparse memory usage (e.g., allocating memory for an OSPF state even if the device only runs BGP) and rigid compile-time coupling. In the upgraded `ank_nte`, devices and interfaces are pure **Graph Nodes/Endpoints** (integer IDs in the graph). Protocol states are stored in contiguous memory arrays (Struct of Arrays).
2. **Mathematical Functor Alignment:** The columnar layout maps perfectly to the thesis's definition of a Functor ($\mathcal{F}_\ell$). Extracting an overlay layer (e.g., $\mathcal{F}_{\text{BGP}}$) is no longer an iterative loop checking `if node.bgp.is_some()`; it is an $O(1)$ memory fetch of the `[BgpState]` attribute vector.
3. **Cypher-like Graph Querying:** The transition to a columnar model invalidates the existing Common Expression Language (CEL) query engine, which relies on dot-notation to traverse static structs. However, because `ank_nte` is already a graph database, the orchestration layer will instead utilise `ank_nte`'s

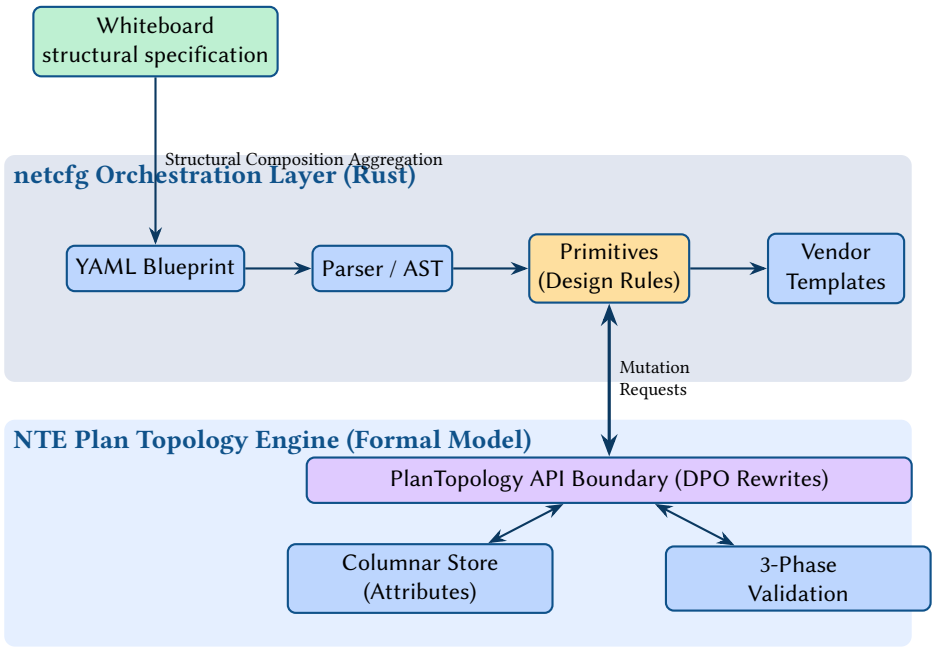

Figure 1. Proposed Two-Layer Architecture integrating the Structural Composition strategy with a formal engine.

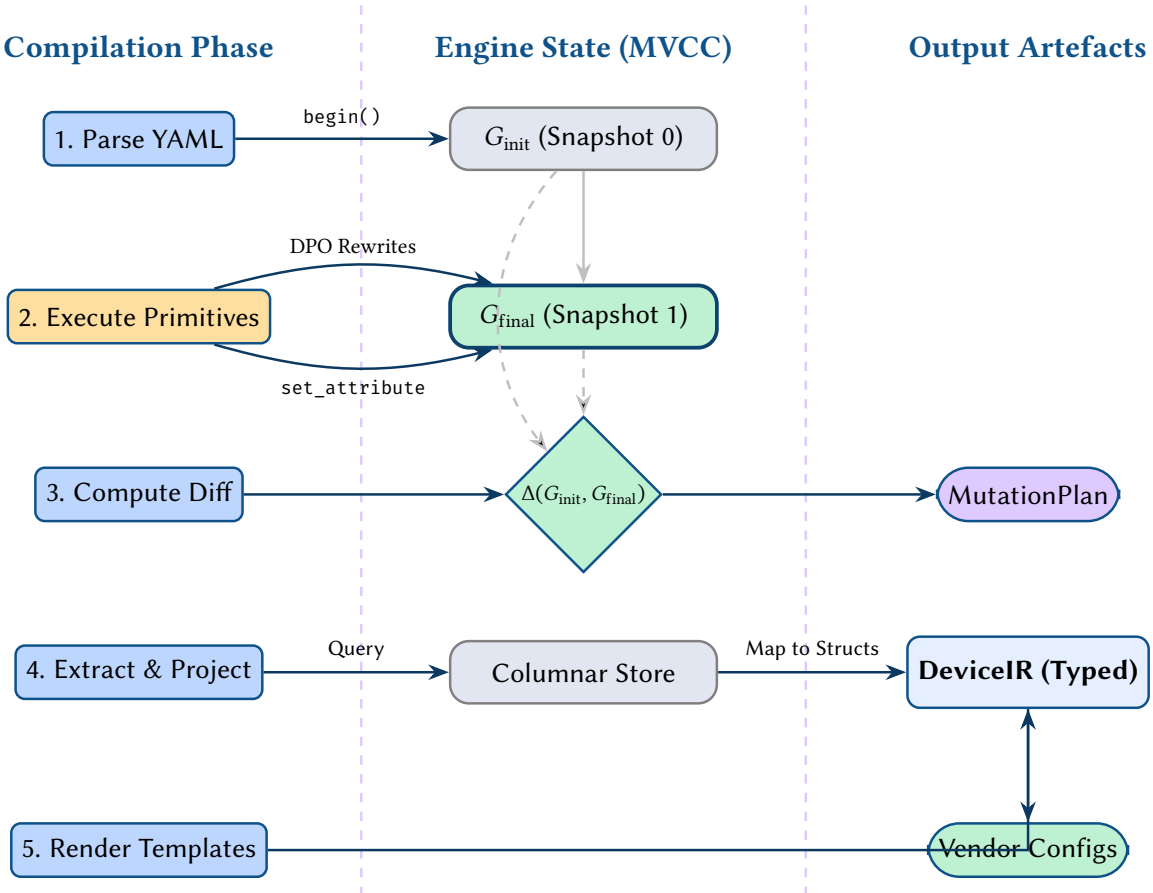

Figure 2. State Lifecycle and Data Flow in the Two-Layer Architecture, illustrating MVCC diffing and the DeviceIR projection gap.

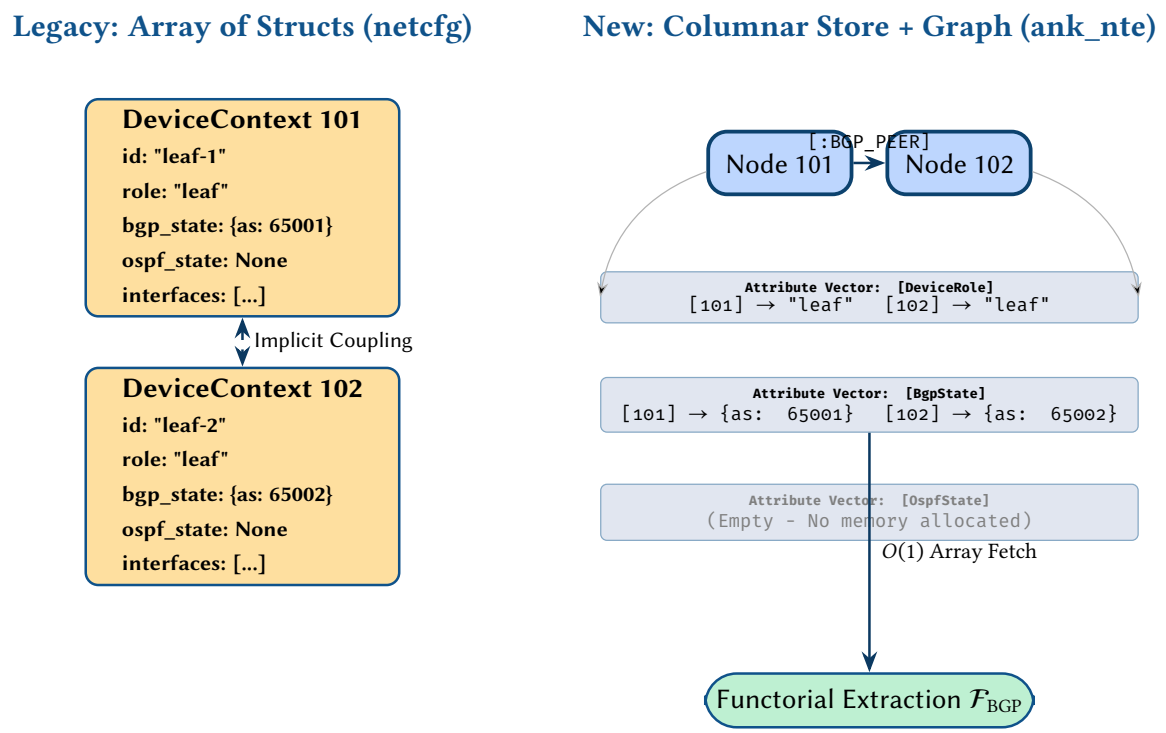

Figure 3. Memory Layout Shift: Moving from rigid `DeviceContext` structs (AoS) to an `ank_nte`-native Columnar Attribute Store (SoA) where graph nodes map to contiguous attribute vectors.

native graph traversal properties (e.g., following E_{conn} edges) to execute declarative, Cypher-like queries (e.g., `MATCH (a)-[:BGP_PEER]->(b)`).

7.3 Detailed Pros

1. **Best of both worlds (Separation of Volatility).** `netcfg`'s proven orchestration (which is highly volatile due to vendor template churn) is isolated from the formal model's rigorous data model (which is mathematically stable).
2. **Clean interface boundary.** The API between layers forces explicit contracts: `netcfg` requests topology mutations via a well-defined interface; the engine validates and applies them.
3. **Incremental migration.** `netcfg` primitives can be migrated one-by-one from direct NTE manipulation to the plan topology API, maintaining a working system throughout.
4. **Thesis narrative.** The two-layer architecture itself is a contribution: it demonstrates how formal specifications can be integrated into existing engineering artefacts without discarding them.
5. **Zero-Compilation Extensibility.** Because the new NTE engine utilises a dynamic columnar store, new protocols are added at the engine layer purely by registering an attribute schema and a functor rule. This completely eliminates the need to author new Rust structs and recompile the compiler just to support a new BGP address family.
6. **Verification path.** The engine layer can natively implement supra-adjacency tensor queries and cross-layer reachability without affecting or bloating the orchestration layer.

7.4 Detailed Cons

1. **The Rendering Gap & Projection Layer.** Option C retains `netcfg`'s MiniJinja templates, which are tightly coupled to the statically-typed `DeviceIR` structs. Bridging the dynamically-typed columnar engine back to `DeviceIR` requires a complex "Projection Layer." This layer must perform an $O(|V| \times |A|)$ serialization step at the end of every compilation, traversing the mathematical attribute state to construct the rigid Rust structs expected by the templates. This impedance mismatch acts as a costly anti-corruption layer.
2. **Diffing Ownership Shift.** If the engine introduces MVCC, the `DiffEngine` should ideally be pushed down into the formal model rather than remaining in the orchestration layer, which increases the implementation effort of the new engine.
3. **API Interface Bifurcation.** The interface between layers must be rich enough to express all primitive operations but narrow enough to enforce invariants. To prevent computational bloat, the API must be explicitly bifurcated (as shown in Figure 4): rigorous DPO rewrites are mandated for structural changes (e.g., adding nodes/edges to G_{plan}), whereas simpler attribute setters are used for trivial property updates. Managing this dual-pathway API complicates primitive authoring.

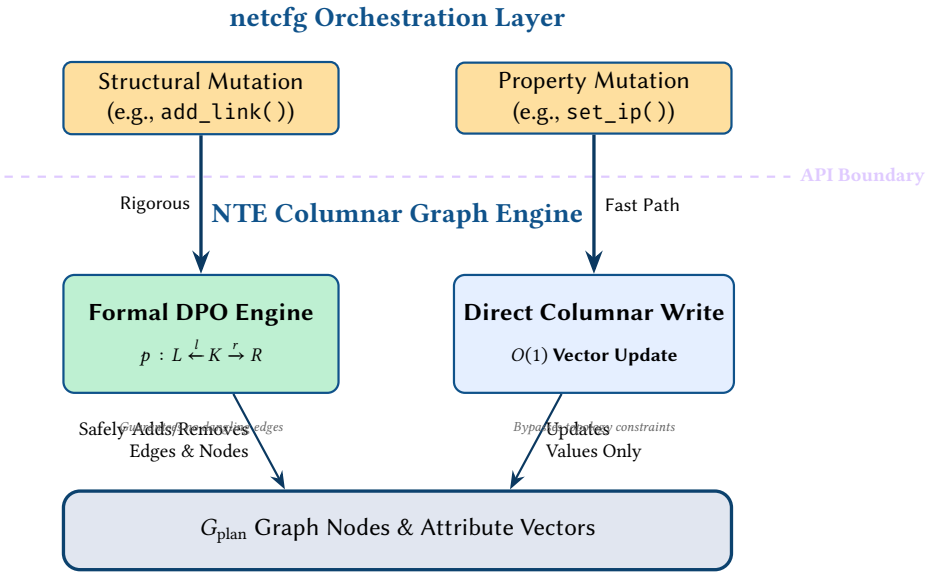

Figure 4. The required API Interface Bifurcation. Structural mutations pass through the rigorous formal DPO engine to guarantee graph invariants, while property mutations use a fast-path direct columnar write to avoid computational bloat.

4. **Query Language Upgrade (CEL to Graph-Native).** The orchestration layer currently relies on a CEL-based NQL (Network Query Language) to filter topologies. CEL predicates assume a fixed memory layout (i.e., fast struct field access). Moving to a dynamic columnar store renders existing NQL macros obsolete. However, because the `ank_nte` engine is already fundamentally a graph database, this presents an opportunity to upgrade the query engine. The orchestration layer must transition from iterative CEL queries to a declarative, Cypher-like graph query mechanism native to `ank_nte` (see Figure 5). While an architectural improvement, rewriting the existing NQL macros requires significant effort.
5. **Performance overhead.** Crossing the layer boundary introduces serialisation and validation costs that direct NTE manipulation avoids.
6. **Provenance Traceability Gap.** When the formal engine rejects a structural mutation (e.g., an invalid DPO rewrite) or an attribute setter fails schema validation, the engine has no knowledge of the YAML AST that triggered it. The orchestration layer must build a "Source Map" or Provenance Tracker to explicitly link graph entity IDs back to the line numbers in the user's intent blueprint, otherwise compiler error messages will be completely opaque to the user.
7. **Transactional Boundary Enforcement.** If a complex primitive (e.g., `build_protocol_layer`) executes a loop of 50 graph mutations and fails on the 49th, it cannot leave the `ank_nte` graph in a partially mutated state. The orchestration layer must strictly wrap every primitive execution in an MVCC transaction (`begin()`, `commit()`, `rollback()`), which requires rigorous error handling in the Rust code to prevent dirty state leakage.
8. **Dual maintenance.** Two layers means two codebases to maintain, test, and version. Breaking changes in the engine API cascade to all primitives.
9. **Partial formal coverage.** DPO rewriting and full functorial composition may still be deferred if the engine API is too coarse; the formal model is only as faithfully implemented as the API allows.
10. **Complexity.** The system has more moving parts than either Option A or Option B; debugging requires understanding both layers and their interaction.

Current: CEL-based NQL (Static Structs)

```
// Select EVPN leaf nodes (Iterative)
nql.filter(
  topology.nodes,
  "node.device_role == 'leaf' &&
   'evpn' in node.protocols"
)

// Find BGP peers (Struct Traversal)
let peers = nql.map(
  node.interfaces,
  "iface.peer_ip"
)
```

Proposed: Graph-Native (Columnar Store)

```
// Match subgraph natively (Declarative)
MATCH (n:Device {role: 'leaf'})
  -[:OWNS]-> (e:Endpoint)
WHERE has_prefix(n, 'evpn')
RETURN n, e

// Traverse multilayer tensor
MATCH (n1:Device)-[:BGP_PEER]->(n2:Device)
RETURN n1.bgp_as, n2.bgp_as
```

Figure 5. Comparison of the existing CEL-based Network Query Language (NQL) which operates on statically-typed Rust structs, versus a proposed Graph-Native query language necessary to query the dynamically-typed columnar plan topology.

7.5 Effort Estimate and Phasing

Phase	Scope	Estimate
H1	Define plan topology API (trait + types)	2–3 weeks
H2	Implement engine: plan topology + attribute store + validation	4–6 weeks
H3	Migrate topology primitives (`mesh_nodes`, `build_protocol_layer`)	2–3 weeks
H4	Migrate allocation primitives (IPAM, resources)	2–3 weeks
H5	Migrate policy primitives (routing, security, QoS)	3–4 weeks
H6	Protocol catalogue + schema enforcement	2–3 weeks
H7	Dependency DAG + ordering validation	1 week
H8	Integration testing + case study validation	2–3 weeks
	Total	**18–26 weeks**

7.6 Risk Assessment

API risk (Medium–High): The interface boundary is the critical design decision; an overly narrow API forces workarounds, while an overly broad API fails to enforce invariants. The necessity of a bifurcated API (DPO + attribute setters) complicates this.

Rendering gap risk (High): The complexity of the "Projection Layer" required to translate the dynamic columnar attribute store back into the static `DeviceIR` format expected by the Jinja templates is a significant unknown.

Compiler UX risk (Medium): If the provenance traceability gap is not solved, the compiler will emit opaque graph engine errors rather than actionable YAML line numbers, severely degrading user experience.

Query engine risk (Medium): Deprecating the CEL-based NQL requires extending `ank_nte` to support Cypher-like traversals. While the underlying graph database architecture is solid, rewriting the extensive library of existing NQL macros is a significant migration effort.

Timeline risk (Medium): 4–6 months is ambitious but feasible within a thesis timeline if scoped carefully.

Integration risk (Medium): Migrating 23 primitives to a new API surface introduces regression potential.

Scope risk (Low–Medium): The phased migration allows stopping at any point with a partially migrated but fully functional system.

7.7 Interface Boundary Definition

The plan topology engine would expose the following core API surface:

1. **Graph operations:** `add_node`, `add_endpoint`, `add_connectivity_edge`, `add_ownership_edge`, `remove_*`, `query_*`.
2. **Attribute operations:** `set_attribute` (with grammar validation), `get_attribute`, `bulk_set` (DataFrame path), `validate_schema`.
3. **Layer operations:** `extract_layer` (functorial extraction), `list_layers`, `layer_dependency_order`.
4. **Validation:** `validate_structural`, `validate_protocol_consistency`, `validate_completeness`.
5. **Transaction:** `begin`, `commit`, `rollback`, `snapshot`.
6. **Mutation (DPO):** `apply_rewrite(pattern_L, glue_K, replacement_R)` — Allows the orchestration layer to submit a topology mutation as a formal DPO rule, guaranteeing no dangling edges and preserving structural invariants during execution.

8 Comparative Evaluation

8.1 Multi-Criteria Decision Matrix

Seven criteria are evaluated on a 1–5 scale (1 = poor, 5 = excellent). Weights reflect thesis priorities: theoretical fidelity and thesis alignment are weighted highest (0.20 each), followed by time-to-value (0.15) and risk (0.15), then implementation effort (0.10), extensibility (0.10), and user impact (0.10).

Criterion	Wt	Option A		Option B		Option C	
		Raw	Wtd	Raw	Wtd	Raw	Wtd
Implementation effort	0.10	5	0.50	1	0.10	3	0.30
Risk	0.15	4	0.60	1	0.15	3	0.45
Theoretical fidelity	0.20	2	0.40	5	1.00	4	0.80
User impact	0.10	5	0.50	2	0.20	4	0.40
Extensibility	0.10	2	0.20	5	0.50	4	0.40
Time-to-value	0.15	5	0.75	1	0.15	3	0.45
Thesis alignment	0.20	3	0.60	4	0.80	5	1.00
Total	1.00		**3.55**		**2.90**		**3.80**

Ranking: Option C (3.80) > Option A (3.55) > Option B (2.90).

8.2 Sensitivity Analysis

The ranking is robust under most weight perturbations:

- **Option A overtakes Option C** only if time-to-value weight increases to ≥ 0.25 *and* theoretical fidelity drops to ≤ 0.10. This scenario corresponds to "just ship it" with no concern for formal rigour—inconsistent with thesis goals.
- **Option B overtakes Option C** only if theoretical fidelity weight increases to ≥ 0.35 *and* all pragmatic criteria (effort, risk, time-to-value) are ignored. This scenario is unrealistic given fixed thesis timelines.
- **Option C remains first** across all weight perturbations where no single criterion exceeds 0.30, confirming it as the balanced choice.

8.3 Ranked Recommendation

1. **Option C: Two-Layer Architecture** (recommended). Achieves the best balance of theoretical fidelity, practical utility, and thesis alignment. The two-layer decomposition is itself a research contribution demonstrating principled integration of formal models with engineering artefacts.
2. **Option A: Incremental Backport** (fallback). If the plan topology engine proves too ambitious within the thesis timeline, Option A provides a safe retreat that still advances the codebase toward formal alignment.
3. **Option B: Clean-Slate Reimplementation** (not recommended). The timeline risk is prohibitive and the expected value is negative: a half-finished rewrite is worse than either a complete backport or a partially migrated two-layer system.

9 Migration Pathways

9.1 Option C Migration: Phase-by-Phase Plan

The recommended migration proceeds in three stages, each delivering a self-contained improvement.

9.1.1 Stage 1: Foundation (Weeks 1–6)

1. **API Proof of Concept (PoC):** Before full engine implementation, draft the `PlanTopology` trait and attempt to port one complex primitive (e.g., `build_protocol_layer` or `overlay_evpn`) against a mock implementation to empirically validate the API boundary and DPO rewrite feasibility.
2. Define the `PlanTopology` trait in `ank_nte` with core graph operations (add/remove/query for nodes, endpoints, connectivity edges, ownership edges).
3. **MVCC Transaction Wrapper:** Implement strict `begin()`, `commit()`, and `rollback()` semantics on the engine API to ensure primitives cannot leave the graph in a corrupted state upon failure.
4. Implement the attribute store with grammar validation: every `set_attribute` call checks the key against the EBNF grammar and the value against the type registry.
5. **Provenance Tracker:** Extend the parser to build a Source Map linking AST line numbers to generated graph Entity IDs, ensuring validation errors can be traced back to user intent.
6. Implement structural validation (Phase 1 of the formal model): unique ownership, no self-loops, bidirectional connectivity.
7. Write integration tests demonstrating that the existing NTE graph can be wrapped in the `PlanTopology` interface.

Decision gate: If the trait API cannot express the operations needed by `mesh_nodes` and `build_protocol_layer` without escape hatches, reconsider the API design before proceeding. If the API is fundamentally incompatible, fall back to Option A.

9.1.2 Stage 2: Primitive Migration & Query Engine (Weeks 7–18)

1. Migrate topology-tier primitives (`mesh_nodes`, `build_protocol_layer`) to use the bifurcated `PlanTopology` API (DPO for structure, setters for attributes).

2. **Query Engine Refactor:** Extend `ank_nte`'s graph database capabilities to support declarative, Cypher-like queries, and rewrite existing CEL-based NQL macros to use this new interface.
3. Migrate allocation-tier primitives (`provision_ips`, `define_ip_pool`, etc.) to write attributes via the validated attribute store.
4. **Projection Layer Implementation:** Build the serialisation boundary that queries the columnar store and instantiates frozen `DeviceIR` structs to maintain compatibility with existing MiniJinja templates.
5. Migrate policy-tier primitives to write typed stanzas via the attribute schema, relying on the new Projection Layer for rendering.
6. Implement the dependency DAG and add compile-time ordering validation.
7. After each migration batch, run the full test suite and case study validation to ensure no regressions.

Decision gate: If more than 30% of primitives require escape hatches around the `PlanTopology` API, the interface is too narrow. Widen it before continuing, or pivot to Option A for the remaining primitives.

9.1.3 Stage 3: Formal Features (Weeks 19–26)

1. Implement functorial layer extraction: `extract_layer(`ℓ`)` filters the plan topology by attribute prefix and returns a subgraph.
2. Implement protocol consistency validation (Phase 2): timer agreement, peer reciprocity, IP uniqueness.
3. Implement completeness validation (Phase 3): verify all required attributes are present for active protocol layers.
4. Integrate the protocol catalogue from ANK-TR-2026-04 as the schema source for the attribute type registry.
5. Write the thesis chapter describing the two-layer architecture, including the API design rationale and migration experience.

Decision gate: Functorial extraction must produce identical overlay graphs to the current `build_protocol_layer` output for all case studies. If it diverges, investigate whether the divergence represents a netcfg bug or a model under-specification.

9.2 Option A Fallback: Outline

If the Option C migration stalls at any decision gate, the following Option A backport sequence provides an alternative:

1. Attribute grammar validation on `DeviceContext.metadata` (1–2 weeks).
2. Dependency DAG for primitive ordering (1 week).
3. Three-phase validation systematisation (2–3 weeks).
4. Protocol catalogue as schema registry (2–3 weeks).
5. Document divergent concepts as "future work" in the thesis.

Total: 6–9 weeks, with clear delineation of implemented vs specified concepts.

9.3 Decision Gate Summary

Gate	When	Criterion	Action if Failed
G1	Week 6	PlanTopology API covers topology primitives	Redesign API or fall back to A
G2	Week 18	$\leq$30% primitives need escape hatches	Widen API or fall back to A
G3	Week 26	Functorial extraction matches case studies	Investigate divergence

10 Conclusion

This report has analysed three strategies for reconciling the netcfg compiler with the plan topology model specified in ANK-TR-2026-03/04. The multi-criteria evaluation ranks the **two-layer architecture** (Option C) highest, achieving the best balance between theoretical fidelity, practical utility, and thesis alignment.

The two-layer approach assigns each artefact to the role it fills best: the formal model provides a rigorous columnar plan topology engine with attribute schemas, functorial extraction, and three-phase validation; `netcfg` provides proven orchestration with 23 primitives, 16 CLI commands, and vendor-specific rendering. The interface boundary between layers enforces explicit contracts while allowing incremental migration.

Crucially, this report identifies the critical architectural bridges required to make this integration viable. The migration must account for translating the dynamic columnar store back into static templates via a Projection Layer, upgrading the legacy CEL queries to a Graph-Native traversal engine, and enforcing strict MVCC transaction boundaries and Provenance Tracking to ensure robust compiler UX.

The recommended 26-week migration plan proceeds in three stages with decision gates at weeks 6, 18, and 26. If the migration stalls, Option A (incremental backport) provides a safe fallback requiring only 6–9 additional weeks. Option B (clean-slate rewrite) is not recommended due to prohibitive timeline risk.

Critically, the two-layer architecture is not merely an engineering expedient but a *research contribution*: it demonstrates a principled methodology for integrating formal specifications into existing, production-grade software systems—a challenge common across systems research.

A Formal Concept Mapping

netcfg Concept	**Formal Notation**	**Notes**
`DeviceContext` fields	Attribute map $\pi_V : V \to A_V$	Fixed struct vs flat map
`InterfaceEndpoint`	Endpoint $e \in E$ with $\pi_E(e)$	Embedded in DeviceContext, not first-class graph entity
`DeviceContext.metadata`	Flat attributes $\pi_V(v)$	Untyped `HashMap<String, JsonValue>`
Primitive execution	Functor application $\mathcal{F}_t(G_{\text{plan}})$	Imperative mutation vs categorical mapping
`build_protocol_layer`	Layer extraction V_ℓ, E_ℓ	Clones nodes; formal model filters by prefix
Layer ordering in YAML	Dependency DAG $\mathcal{F}_t \to \mathcal{F}_k$	Manual vs computed
Shadow topology	MVCC snapshot	No true versioning; shadow is read-only copy
NTE-QL selectors	Layer extraction predicates	CEL-based; no formal predicate calculus
`DiffEngine`	Graph differencing $\Delta(G, G')$	Semantic keying; no DPO guarantees
`assert_state`	Protocol consistency rules	User-authored CEL; not exhaustive
Template preflight	Completeness validation	Checks template vars, not attribute schema
`freeze()` → DeviceIR	Immutable snapshot	One-way; no rollback
NTE overlay graphs	Multilayer composition $\bigoplus_\ell$	Separate graphs; no tensor coupling

B Protocol Coverage Gap

The following table compares the protocol attribute prefixes catalogued in ANK-TR-2026-04 against current netcfg support. “Full” indicates the protocol is modelled with dedicated primitives or typed stanzas; “Partial” indicates some attributes are handled via metadata or untyped stanzas; “None” indicates no implementation.

Prefix	Protocol	netcfg Status	Priority
Tier 1: Core Routing & Switching			
`ip`	IPv4/IPv6 addressing	Full	—
`ospf`	OSPF	Full	—
`bgp`	BGP	Full	—
`isis`	IS-IS	Partial	Medium
`vlan`	VLANs	Partial	Medium
`stp`	Spanning Tree	None	Low
`lacp`	Link Aggregation	None	Medium
`bfd`	BFD	None	Medium
`mpls`	MPLS	None	Medium
`sr`	Segment Routing	Partial	High
`evpn`	EVPN	Full	—
`switchport`	L2 port mode	None	Low
Tier 2: Overlay & Multicast			
`vxlan`	VXLAN	Full	—
`gre`	GRE tunnels	None	Low
`ipsec`	IPsec	None	Low
`geneve`	Geneve	None	Low
`pim`	PIM multicast	None	Low
`igmp`	IGMP	None	Low
`rsvp`	RSVP-TE	None	Low
`ldp`	LDP	None	Low
Tier 3: Physical, Security, Management			
`phy`	Physical layer	None	Low
`dom`	Digital optical monitoring	None	Low
`acl`	Access control lists	Full	—
`qos`	Quality of Service	Full	—
`macsec`	MACsec	None	Low
`dot1x`	802.1X	None	Low
`sgt`	Security Group Tags	None	Low
`lldp`	LLDP	None	Low
`vrrp`	VRRP	Partial	Medium
`hsrp`	HSRP	Partial	Medium

Summary: Of the 30 representative protocol prefixes listed above, netcfg has full support for 7 (23%), partial support for 6 (20%), and no support for 17 (57%). Tier 1 coverage is the strongest (7/12 partial or better); Tier 2 and Tier 3 are largely unimplemented. The priority column reflects relevance to the current case study portfolio; low-priority protocols can be deferred to post-thesis milestones.